DCHL Thinking Machine

Irfan Mahir — Independent Researcher, Dhaka, Bangladesh — irfan@furylogic.com

Contents

DCHL Thinking Machine **1**
- Quick Start . . . 1
- What It Is . . . 1
  - What It Can Do (Same Tasks as ML/DL — Cheaper, CPU Only) . . . 1
  - What It Is NOT . . . 2
- Architecture . . . 2
  - Training vs Inference . . . 2
- Hardware . . . 3
  - vs The Industry . . . 3
- Files . . . 3
- CLI . . . 3
  - Training (`lto`) . . . 3
  - Chat (`chatbot`) . . . 3
- Research . . . 3
- License . . . 4

> **STATUS: RETIRED ARCHITECTURE.** This folder documents the DCHL substrate, which was replaced by the oscillator substrate. It is kept because the negative result is a contribution — SI paper §2 is a self-critique built on it. Only RIFA (IC/SIL) remains load-bearing. See SI paper §6.2.2. Papers in `DEAD/` are formally dead: §6.2.3.

DCHL Thinking Machine

Irfan Mahir *Independent Researcher, Dhaka, Bangladesh* irfan@furylogic.com

Decoupled Contrastive Hebbian Learning — a physics-based cognitive architecture that *thinks* rather than *generates*. No tokens, no attention, no backprop, no GPU needed.

Input → byte-mapped cognitive state (256 floats) → Hamiltonian reasoning on learned potential landscape → deeper understanding (lower Φ).

Quick Start

```
make -B chatbot lto      # build (CPU only, g++ 9+)
./lto --epochs 200 --save chatbot.neuron  # train ~3 min
./chatbot chatbot.neuron                  # think interactively
```

```
=== DCHL Thinking Machine ===
Input → cognitive state → Hamiltonian reasoning
The state IS the understanding.
```

```
> what is the capital of france?
  Φ(initial)=-295.1  conf=0.999  ood=0
  Φ(final)=-737.3    ΔΦ=-442.2  depth=100
  ✓ Deepened understanding
```

What It Is

A **thinker**, not a generator. The model compresses text directly into a 256-float cognitive state (byte→float, no tokenizer), then evolves that state through a learned energy landscape via Hamiltonian dynamics. The evolved state IS the understanding — not a text string, not a probability distribution over tokens, but a compressed cognitive representation.

What It Can Do (Same Tasks as ML/DL — Cheaper, CPU Only)

Task	How	vs ML/DL	vs LLM
Classification	Lightweight head on evolved state	No GPU needed	No GPU needed
Similarity	Cosine between cognitive states	No embedding model	No embedding model
Anomaly	Φ(c) IS the anomaly score — built-in	No OOD training	No OOD training
Retrieval	State-to-state nearest neighbor	No index needed	No KV-cache needed
Reasoning	Hamiltonian dynamics → deeper understanding	No chain-of-thought	No LLM inference

What It Is NOT

- × Not a text generator (doesn't produce output tokens)
- × Not a transformer (no attention, no KV-cache, no context window)
- × Not an LLM (33K params, not billions)
- × Not gradient-based (no backprop, no loss functions, no Adam)
- × Not a statistical pattern matcher (Hamiltonian mechanics on learned energy landscape)

Architecture

```
State (256 floats) → Hidden (128, tanh) → Output (1 float = Φ)
        ↑                                        ↑
    byte→float mapping                    energy of cognitive state
    (no tokenizer)                       (lower = more "understood")
```

Two independent learning processes, both local, no backprop: - **Hidden layer**: per-unit contrastive Hebbian — units differentiate text from random - **Output layer**: pure differential projection — maps features to energy values

Training is text-vs-random: $\Phi(\text{text}) < \Phi(\text{random})$, that's all. No transitions, no predictions, no targets.

Training vs Inference

```
Training:
  For each pair (text, random):
    Φ(text)  → push lower (h_pos - h_neg)
    Φ(random) → push higher (h_neg - h_pos)

Inference (Hamiltonian reasoning):
  c₀ = byte→float(input)
  for step in 1..100:
    p += -∇Φ(c) · dt      (gradient of potential = force)
    c += p · dt           (momentum drives state)
  c_final = evolved, more "understood" state
  ΔΦ = Φ(c_final) - Φ(c₀)  (negative = understanding deepened)
```

Hardware

Runs on commodity CPU servers. The same old hardware used for spam detection.

- No GPU required
- ~33K parameters (fits in L2 cache)
- Training: ~3 minutes for 200 epochs
- Inference: ~0.01s for 100 reasoning steps
- Model file: ~264 KB

vs The Industry

Architecture	Params	Hardware	Training	Inference
DCHL (this)	33K	Any CPU	~3 min	~0.01s
BERT-base	110M	GPU	days	~0.1s
GPT-3	175B	H100 cluster	months	~1s
Llama 3 70B	70B	H100 cluster	weeks	~0.5s

DCHL inverts the scaling law: better physics → deeper understanding — same compute. Not bigger models → diminishing returns.

Files

File	Purpose
`src/hcma.hpp`	DCHL potential, contrastive Hebbian updates, Hamiltonian dynamics
`src/lto.cpp`	Training entry point (CPU/GPU, CLI, eval)
`src/chatbot.cpp`	Interactive thinking machine
`src/bench.cpp`	Performance benchmarking
`src/gpu_training.cu`	CUDA kernels (optional)

CLI

Training (`lto`)

```
./lto --epochs 200 --lr-hidden 0.01 --lr-output 0.1 --save model.neuron
```

Key flags: `--epochs`, `--lr-hidden`, `--lr-output`, `--d-state`, `--d-hidden`, `--data`

Chat (`chatbot`)

```
./chatbot model.neuron --model potential.neuron --steps 100 --dt 0.005
```

Shows Φ(initial), Φ(final), ΔΦ, confidence, OOD flag.

Research

- HCMA paper: Full architecture & math
- Implementation notes: Every problem & fix

License

MIT